

```
In [2]: import pandas as pd
df = pd.read_csv(r"E:\pradnya's programming\Domains\ML\DA_clgAss\train_u6lujuX_C
df.head()
```

```
Out[2]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0	Graduate	No	5849
1	LP001003	Male	Yes	1	Graduate	No	4583
2	LP001005	Male	Yes	0	Graduate	Yes	3000
3	LP001006	Male	Yes	0	Not Graduate	No	2583
4	LP001008	Male	No	0	Graduate	No	6000

```
In [3]: print(df.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Loan_ID               614 non-null   object  
1   Gender                601 non-null   object  
2   Married               611 non-null   object  
3   Dependents            599 non-null   object  
4   Education             614 non-null   object  
5   Self_Employed         582 non-null   object  
6   ApplicantIncome       614 non-null   int64   
7   CoapplicantIncome     614 non-null   float64  
8   LoanAmount            592 non-null   float64  
9   Loan_Amount_Term      600 non-null   float64  
10  Credit_History        564 non-null   float64  
11  Property_Area         614 non-null   object  
12  Loan_Status           614 non-null   object  
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None
```

```
In [4]: # hence , df contains null values
print(df.isnull().sum())
```

```
Loan_ID      0
Gender       13
Married       3
Dependents   15
Education     0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount   22
Loan_Amount_Term 14
Credit_History 50
Property_Area 0
Loan_Status  0
dtype: int64
```

Handling Missing Values

- Deleting missing values

```
In [5]: # 1) delete rows with missing data
new_df = df.dropna(axis=0)
print("Old data frame length:", len(df))
print("New data frame length:", len(new_df))
```

Old data frame length: 614

New data frame length: 480

```
In [6]: #2) try deleting column with higher null insted
new_df2 = df.drop(['Credit_History'],axis=1)
new_df2 = new_df2.dropna(axis=0)
print("Old data frame length:", len(df))
print("New data frame length after deleting column:", len(new_df2))
```

Old data frame length: 614

New data frame length after deleting column: 523

- Replacing Missing Values

```
In [7]: #3) filling missing value with mean
df['ApplicantIncome'] = df['ApplicantIncome'].fillna(df['ApplicantIncome'].mean())
df['CoapplicantIncome'] = df['CoapplicantIncome'].fillna(df['CoapplicantIncome'].mean())
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mean())
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mean())
df.head()
print(df.isnull().sum())
```

```
Loan_ID      0
Gender       13
Married       3
Dependents   15
Education     0
Self_Employed 32
ApplicantIncome  0
CoapplicantIncome  0
LoanAmount    0
Loan_Amount_Term  0
Credit_History  0
Property_Area  0
Loan_Status   0
dtype: int64
```

```
In [8]: # now Let's handle categorical variables
df['Gender'].value_counts()
null_rows = df.query('Gender != Gender')

'''The condition 'Gender != Gender' is a clever way to identify null values,
as NaN is not equal to itself.'''
print(null_rows)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
23	LP001050	NaN	Yes	2	Not Graduate	No	
126	LP001448	NaN	Yes	3+	Graduate	No	
171	LP001585	NaN	Yes	3+	Graduate	No	
188	LP001644	NaN	Yes	0	Graduate	Yes	
314	LP002024	NaN	Yes	0	Graduate	No	
334	LP002103	NaN	Yes	1	Graduate	Yes	
460	LP002478	NaN	Yes	0	Graduate	Yes	
467	LP002501	NaN	Yes	0	Graduate	No	
477	LP002530	NaN	Yes	2	Graduate	No	
507	LP002625	NaN	No	0	Graduate	No	
576	LP002872	NaN	Yes	0	Graduate	No	
588	LP002925	NaN	No	0	Graduate	No	
592	LP002933	NaN	No	3+	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
23	3365	1917.0	112.0	360.0	
126	23803	0.0	370.0	360.0	
171	51763	0.0	700.0	300.0	
188	674	5296.0	168.0	360.0	
314	2473	1843.0	159.0	360.0	
334	9833	1833.0	182.0	180.0	
460	2083	4083.0	160.0	360.0	
467	16692	0.0	110.0	360.0	
477	2873	1872.0	132.0	360.0	
507	3583	0.0	96.0	360.0	
576	3087	2210.0	136.0	360.0	
588	4750	0.0	94.0	360.0	
592	9357	0.0	292.0	360.0	

	Credit_History	Property_Area	Loan_Status
23	0.000000	Rural	N
126	1.000000	Rural	Y
171	1.000000	Urban	Y
188	1.000000	Rural	Y
314	1.000000	Rural	N
334	1.000000	Urban	Y
460	0.842199	Semiurban	Y
467	1.000000	Semiurban	Y
477	0.000000	Semiurban	N
507	1.000000	Urban	N
576	0.000000	Semiurban	N
588	1.000000	Semiurban	Y
592	1.000000	Semiurban	Y

```
In [9]: df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
print(df['Gender'].loc[null_rows.index])
# print(df["Gender"].mode())
```

```

23      Male
126     Male
171     Male
188     Male
314     Male
334     Male
460     Male
467     Male
477     Male
507     Male
576     Male
588     Male
592     Male
Name: Gender, dtype: object

```

```

In [10]: df['Married'].value_counts()
null_rows = df.query('Married != Married')

print(null_rows)
# df.head()
df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
# print(null_rows)

```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
104	LP001357	Male	NaN	NaN	Graduate	No	
228	LP001760	Male	NaN	NaN	Graduate	No	
435	LP002393	Female	NaN	NaN	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
104	3816	754.0	160.000000	360.0	
228	4758	0.0	158.000000	480.0	
435	10047	0.0	146.412162	240.0	

	Credit_History	Property_Area	Loan_Status
104	1.0	Urban	Y
228	1.0	Semiurban	Y
435	1.0	Semiurban	Y

```

In [11]: print(df['Married'].loc[null_rows.index])
# df.head()
print(df.isnull().sum())

```

```

104      Yes
228      Yes
435      Yes
Name: Married, dtype: object
Loan_ID      0
Gender       0
Married      0
Dependents   15
Education    0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount    0
Loan_Amount_Term 0
Credit_History 0
Property_Area  0
Loan_Status   0
dtype: int64

```

```
In [12]: # Set display options to show all rows
pd.set_option('display.max_rows', None)

# # Print the entire 'Dependents' column
print(df['Dependents'].to_string(index=False))
```

0
1
0
0
0
2
0
3+
2
1
2
2
2
0
2
0
1
0
0
0
0
1
0
2
1
0
0
2
0
2
1
0
1
0
3+
0
0
0
0
0
0
0
0
0
0
0
0
0
0
1
0
0
0
0
0
2
1
2
0
0
1
2

0
3+
0
1
0
0
0
1
3+
0
0
2
0
3+
3+
0
0
1
3+
3+
0
1
2
0
1
0
2
0
0
0
0
0
2
2
0
0
0
0
0
0
0
0
2
0
NaN
0
NaN
1
2
0
2
3+
0
0
0
1
0
1
0
1
0
0

NaN

0

0

2

0

0

3+

0

1

0

0

0

0

0

0

3+

0

2

0

2

2

0

0

0

2

0

2

1

0

0

0

0

0

2

0

3+

1

1

0

0

0

0

1

2

0

0

0

0

0

2

0

3+

3+

0

0

0

2

3+

1

0

1
0
0
1
0
0
1
0
0
0
0
0
0
0
0
0
1
0
0
0
0
1
2
3+
1
2
0
0
0
0
0
0
3+
1
3+
0
3+
0
0
2
2
0
2
0
0
0
0
NaN
2
NaN
0
1
0
0
0
1
1
0
0
1
1

2
1
0
2
0
0
2
1
1
1
0
0
2
0
1
0
3+
0
3+
0
3+
1
0
1
0
0
0
2
3+
0
1
0
0
0
0
2
1
0
0
0
0
1
0
2
0
0
0
0
0
0
0
0
2
2
NaN
0
3+
1
1
0
1

0
NaN
0
1
0
0
0
0
0
2
0
0
0
2
0
1
2
0
1
1
0
3+
2
0
3+
1
0
0
0
0
1
0
NaN
0
1
NaN
1
2
3+
0
3+
0
1
3+
2
0
NaN
2
0
0
0
0
3+
0
0
NaN
2
1
0
3+

1
2
0
0
0
0
0
0
1
0
0
2
2
1
0
0
3+
0
0
2
0
0
0
0
2
1
0
0
0
0
3+
3+
0
2
2
2
0
0
0
0
2
0
0
0
0
0
0
0
1
3+
1
0
0
0
0
0
1
2
0
0

0
0
0
1
0
0
1
2
0
0
1
0
0
0
0
NaN
0
0
0
0
0
0
0
3+
1
0
1
2
0
2
1
2
2
0
0
0
2
0
0
2
0
0
0
3+
0
1
0
0
3+
0
2
0
1
1
3+
0
2
2
2
2
1
2

0
3+
0
0
2
1
2
1
2
0
0
1
0
0
0
1
0
0
1
0
0
0
2
1
0
2
0
0
0
1
0
1
2
0
0
3+
2
NaN
0
0
2
0
3+
2
0
2
0
1
1
0
0
3+
2
1
0
0
0
2
0
3+

1
0
1
1
0
0
0
0
0
0
2
1
1
0
0
1
0
3+
0
0
2
1
0
0
2
0
0
3+
0
0
1
NaN
2
2
3+
2
0
0
1
0
2
0
0
1
1
1
1
0
0
0
2
0
2
3+
0
0
0
2
NaN
0
2

```
3+
0
3+
0
1
0
1
2
0
0
3+
1
2
0
```

```
In [13]: # convert to integer
# Clean the 'Dependents' column by removing all non-numeric characters
df['Dependents'] = df['Dependents'].astype(str).replace(r'[^\d-]', '', regex=True)
# In this updated code, the regular expression r'[^\d-]' matches any character t
# Convert the cleaned values back to numeric type
df['Dependents'] = pd.to_numeric(df['Dependents'], errors='coerce')
df['Dependents'].head()
```

```
Out[13]: 0    0.0
1    1.0
2    0.0
3    0.0
4    0.0
Name: Dependents, dtype: float64
```

```
In [14]: df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mean())
print(df['Dependents'].isnull().sum())
```

```
0
```

```
In [18]: df['Self_Employed'].value_counts()
null_rows = df.query('Self_Employed != Self_Employed')
df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
print(df['Self_Employed'].loc[null_rows.index])
print(df['Dependents'].isnull().sum())
```

```
Series([], Name: Self_Employed, dtype: object)
0
```

Part B

Outliers

- step 1: Detecting Outliers

```
In [19]: # find outliers
# create a histogram

import plotly.express as px

# # Assuming df is a predefined DataFrame
```



```

# fig = px.histogram(df, x='Dependents')

# fig.show()

# create a function to find outliers using IQR

def find_outliers_IQR(df):

    q1=df.quantile(0.25)

    q3=df.quantile(0.75)

    IQR=q3-q1

    outliers = df[((df<(q1-1.5*IQR)) | (df>(q3+1.5*IQR)))]

    return outliers

```

```

In [20]: q1=df['ApplicantIncome'].quantile(0.25)

q3=df['ApplicantIncome'].quantile(0.75)

IQR=q3-q1

outliers = df[(df['ApplicantIncome'] < (q1 - 1.5 * IQR)) | (df['ApplicantIncome']

print(outliers.shape) # number of outliers

```

(50, 13)

```

In [21]: # using box plot
fig = px.box(df, y='Dependents')

fig.show()

```

```

In [22]: print(find_outliers_IQR(df['Dependents']).shape)

```

(51,)

```

In [23]: fig = px.box(df,y= 'Education')
fig.show()

```

```

In [24]: fig = px.box(df,y= 'ApplicantIncome')
fig.show()

```

```

In [25]: print(find_outliers_IQR(df['ApplicantIncome']).shape)

```

(50,)

, we will calculate the outlier data points using the statistical method called interquartile range (IQR) instead of using Z-score. Using the IQR, the outlier data points are the ones falling below $Q1 - 1.5 \text{ IQR}$ or above $Q3 + 1.5 \text{ IQR}$. The $Q1$ is the 25th percentile and $Q3$ is the 75th percentile of the dataset, and IQR represents the interquartile range calculated by $Q3 - Q1$.

Using the convenient pandas `.quantile()` function, we can create a simple Python function that takes in our column from the dataframe and outputs the outliers:

Notice using `.quantile()` we can define Q1 and Q3. Next we calculate IQR, then we use the values to find the outliers in the dataframe. Since it takes a dataframe, we can input one or multiple columns at a time.